

PROPUESTA DE PROYECTO: OPTIMIZACIÓN DE LOGÍSTICA Y DISTRIBUCIÓN

1. Contexto del Problema

En el entorno competitivo de la logística urbana, la eficiencia en la “última milla” es crucial para la sostenibilidad financiera y la satisfacción del cliente. La empresa **Logística Distrital S.A.** enfrenta el reto de distribuir suministros tecnológicos críticos en la ciudad de Bogotá, específicamente en las localidades de Usaquén, Chapinero y Kennedy.

El problema radica en que la demanda de entregas suele superar la capacidad inmediata, y los tiempos de desplazamiento varían significativamente debido a la infraestructura vial. Se requiere un modelo que permita tomar decisiones informadas sobre la asignación de la flota vehicular para cumplir con compromisos contractuales de entrega mínima mientras se minimizan los costos operativos representados por el tiempo en ruta.

2. Planteamiento del Problema

Se deben distribuir paquetes utilizando una flota limitada de **10 camiones**. La operación se divide en tres zonas principales con características distintas de tiempo y capacidad de entrega. El objetivo principal es determinar la asignación óptima de vehículos que minimice el tiempo total de operación, enfocado para Programación Lineal, o maximice el beneficio de entregas, enfocado para Programación Dinámica, garantizando el cumplimiento de las metas mínimas por zona.

Zona (i)	Nombre	Tiempo de Viaje (t_i)	Paquetes por Vehículo (p_i)	Demanda Mínima (d_i)
1	Usaquén	60 min	15	45
2	Chapinero	45 min	10	30
3	Kennedy	90 min	20	60

3. Modelo Matemático: Método de la Gran M

Este modelo busca la optimización global bajo restricciones de desigualdad.

En este contexto el modelo tiene el objetivo de minimizar tiempos de entrega cumpliendo con el número de entregas mínimas representadas como las restricciones.

Variables de decisión

- x_i : número de vehículos asignados a la zona i , donde $i = 1, 2, 3$.
- a_i : variables artificiales para penalizar el incumplimiento de la demanda mínima.

Función objetivo

$$\text{mín } Z = 60x_1 + 45x_2 + 90x_3 + M(a_1 + a_2 + a_3)$$

Restricciones

$x_1 + x_2 + x_3 \leq 10$	Disponibilidad de flota
$15x_1 + a_1 \geq 45$	Demanda Zona 1
$10x_2 + a_2 \geq 30$	Demanda Zona 2
$20x_3 + a_3 \geq 60$	Demanda Zona 3
$x_i, a_i \geq 0$	No negatividad

4. Modelo Matemático: Programación Dinámica

Este modelo divide el problema en decisiones secuenciales por zonas.

El objetivo es maximizar los paquetes entregados a cada zona respetando la cantidad mínima que se debe entregar por zona

Definiciones

- **Etap**a (n): representa cada zona a la que se le asignarán recursos, con $n = 1, 2, 3$.
- **Estado** (s_n): número de vehículos disponibles para ser asignados en la etapa n y etapas posteriores.
- **Decisión** (x_n): número de vehículos asignados específicamente a la zona n .

Ecuación de recurrencia

Para maximizar el beneficio total, medido como paquetes entregados:

$$f_n(s_n) = \max\{p_n(x_n) + f_{n+1}(s_n - x_n)\}$$

Sujeto a:

$$0 \leq x_n \leq s_n$$

5. Desarrollo de Software

Esta sección presenta la documentación de los algoritmos implementados para resolver y analizar el problema de optimización: el Método de la Gran M y los modelos de Programación Dinámica.

5.1. Método de la Gran M (MetodoDeLaGranM.py)

5.1.1. Propósito

El archivo implementa un solucionador de programación lineal usando el método simplex con la técnica de la Gran M.

5.1.2. Flujo general

- **Datos de entrada:** maximizar (booleano del sentido del problema), `norig` (variables originales $x_1, \dots, x_n$), `fostr` (cadena con la función objetivo, por ejemplo $60x_1 + 45x_2 + 90x_3$) y `constraint_lines` (restricciones en texto, por ejemplo $x_1 + x_2 + x_3 \leq 10$).
- **Parser de expresiones y restricciones:** `parse_expr` extrae coeficientes; `parse_constraint` detecta operadores ($\leq$, $\geq$, $=$) y convierte el lado derecho a fracción. También normaliza lados derechos negativos invirtiendo el signo y el operador.
- **Construcción de variables auxiliares:** agrega holguras s para $\leq$, excesos e y artificiales a para $\geq$, y artificiales a para igualdades.
- **Tabla inicial:** genera el tableau simplex con `n_vars` y `n_rows`. El vector c_j tiene los coeficientes de la función objetivo y las artificiales reciben una penalización M (`M_VALUE = 1_000_000`) para forzar su eliminación.
- **Iteraciones simplex:** calcula z_j y $c_j - z_j$, elige la columna pivote con el mayor $c_j - z_j$, aplica la regla del mínimo cociente y normaliza la fila pivote para actualizar la base.
- **Condiciones de parada:** si `best_czj` $\leq 1e-9$, se alcanza el óptimo; si no existe fila pivote válida, el problema es no acotado; si se supera `max_iter = 100`, retorna `max_iter`.
- **Comprobación de factibilidad:** si alguna variable artificial básica tiene $b > 0$, el problema se considera infactible.

5.1.3. Resultados devueltos

La función principal `solve_gran_m(...)` retorna un par compuesto por:

- **iterations:** lista de iteraciones completas con los datos del tableau, variables básicas, costos básicos, c_j , z_j , $c_j - z_j$, vector b , valor de z e información del pivote.

- **result**: diccionario con el estado final y la solución.

Campos principales de **result**:

- **status**: *optimal*, *infeasible*, *unbounded* o *max_iter*.
- **sol**: valores de las variables originales cuando hay solución óptima.
- **z_real**: valor óptimo ajustado según maximización o minimización.
- **slack_sol**: valores de holguras o excesos.
- **orig_vars**: nombres de variables originales.
- **maximizar**: booleano con el sentido del problema.

5.1.4. Interpretación de resultados

- **optimal**: el problema tiene una solución óptima factible.
- **unbounded**: se encontró una dirección de mejora infinita.
- **infeasible**: no existe una solución factible, debido a variables artificiales no eliminables.
- **max_iter**: el algoritmo no convergió antes del límite de iteraciones.

5.1.5. Puntos críticos

Las variables artificiales se penalizan con $-M$ para forzarlas fuera de la base. El método trabaja con fracciones exactas usando **fractions.Fraction**, lo que reduce errores numéricos. El código asume que la función objetivo y las restricciones son lineales en $x_1, \dots, x_n$.

5.1.6. Muestra de prueba del software

A continuación se presenta una prueba visual del flujo del software para resolver un problema mediante el Método de la Gran M.

Inicio del método: se ingresan los datos iniciales del modelo de programación lineal.

CONFIGURACIÓN

Tipo: Minimizar

Variables de decisión: 3

Nº de restricciones: 4

Generar campos

FUNCIÓN OBJETIVO

Ejemplo: $7x_1 + 7x_2 + 7x_3$

$60x_1 + 45x_2 + 90x_3$

RESTRICCIONES

Ejemplo: $6x_1 + 3x_2 \leq 96$ | Operadores: $\leq$ $\geq$ $=$

R1: $x_1 + x_2 + x_3 \leq 10$

R2: $15x_1 \geq 45$

R3: $10x_2 \geq 30$

R4: $20x_3 \geq 60$

Iteración 0: se construye la tabla inicial con variables de holgura, exceso y artificiales.

ITERACIÓN 0												
		x1	x2	x3	s1	e2	e3	e4	a2	a3	a4	b _j
C _j		-60	-45	-90	0	0	0	0	-1000000	-1000000	-1000000	
VB	CB											
s1	0	1	1	1	1	0	0	0	0	0	0	10
a2	-1000000	15	0	0	0	-1	0	0	1	0	0	45
a3	-1000000	0	10	0	0	0	-1	0	0	1	0	30
a4	-1000000	0	0	20	0	0	0	-1	0	0	1	60
Z _j		-15000000	-10000000	-20000000	0	1000000	1000000	1000000	-1000000	-1000000	-1000000	-135000000

Iteración 1: se selecciona el primer pivote para mejorar el valor de la función objetivo.

ITERACIÓN 1												
↳ Entra: x3 Sale: a4												
		x1	x2	x3	s1	e2	e3	e4	a2	a3	a4	bj
Cj		-60	-45	-90	0	0	0	0	-1000000	-1000000	-1000000	
VB	CB											
s1	0	1	1	0	1	0	0	0.05	0	0	-0.05	7
a2	-1000000	15	0	0	0	-1	0	0	1	0	0	45
a3	-1000000	0	10	0	0	0	-1	0	0	1	0	30
x3	-90	0	0	1	0	0	0	-0.05	0	0	0.05	3
Zj		-15000000	-10000000	-90	0	1000000	1000000	4.5	-1000000	-1000000	-4.5	-75000270

Iteración 2: se actualiza la tabla y se continúa eliminando variables artificiales.

ITERACIÓN 2												
↳ Entra: x1 Sale: a2												
		x1	x2	x3	s1	e2	e3	e4	a2	a3	a4	bj
Cj		-60	-45	-90	0	0	0	0	-1000000	-1000000	-1000000	
VB	CB											
s1	0	0	1	0	1	0.0667	0	0.05	-0.0667	0	-0.05	4
x1	-60	1	0	0	0	-0.0667	0	0	0.0667	0	0	3
a3	-1000000	0	10	0	0	0	-1	0	0	1	0	30
x3	-90	0	0	1	0	0	0	-0.05	0	0	0.05	3
Zj		-60	-10000000	-90	0	4	1000000	4.5	-4	-1000000	-4.5	-30000450

Iteración 3: se realiza el último ajuste de la tabla antes de alcanzar la solución óptima.

ITERACIÓN 3												
↳ Entra: x2 Sale: a3												
		x1	x2	x3	s1	e2	e3	e4	a2	a3	a4	bj
Cj		-60	-45	-90	0	0	0	0	-1000000	-1000000	-1000000	
VB	CB											
s1	0	0	0	0	1	0.0667	0.1	0.05	-0.0667	-0.1	-0.05	1
x1	-60	1	0	0	0	-0.0667	0	0	0.0667	0	0	3
x2	-45	0	1	0	0	0	-0.1	0	0	0.1	0	3
x3	-90	0	0	1	0	0	0	-0.05	0	0	0.05	3
Zj		-60	-45	-90	0	4	4.5	4.5	-4	-4.5	-4.5	-585

Solución: el software muestra el estado final, las variables óptimas y el valor de la función objetivo.

$Z^* = 585$ (MÍNIMO)

Variables de decisión:

x_1

=

3

x_2

=

3

x_3

=

3

Holguras / Excesos:

s_1

=

1

e_2

=

0

e_3

=

0

5.2. Programación Dinámica (ProgramacionDinamica.py)

5.2.1. Propósito

Este módulo ofrece varios modelos de programación dinámica para problemas de optimización discretos.

5.2.2. Estructura general

La clase `ModeloPD` define métodos estáticos para `mochila_01`, `mochila_ilimitada`, `tabla_personalizada` y `camino_dag`. Cada método construye una tabla `dp`, reconstruye la solución y genera pasos de seguimiento.

5.2.3. Método `mochila_01`

La recurrencia compara no incluir el ítem con incluirlo cuando la capacidad lo permite:

$$dp[i][w] = \max\{dp[i-1][w], dp[i-1][w - p_i] + v_i\}$$

Su salida incluye `optimo`, `seleccionados`, `peso_usado`, `dp`, `pasos` y `factible`. Soporta tipo `max` y tipo `min`, permite mínimos por ítem y reconstruye la solución comparando filas sucesivas.

5.2.4. Método `mochila_ilimitada`

La recurrencia evalúa reutilizar ítems mientras el peso no exceda la capacidad:

$$dp[w] = \max_i\{dp[w - p_i] + v_i\}$$

Su salida incluye `optimo`, `usados`, `dp`, `pasos` y `factible`. El estado es unidimensional, por lo que reduce memoria a $O(W)$, e incluye seguimiento `from[w]` para reconstruir los ítems usados.

5.2.5. Método `tabla_personalizada`

La recurrencia general por etapas puede expresarse como:

$$f[e][s] = \max / \min\{f[e-1][s'], r[e][s]\}, \quad s' \geq s$$

Su salida incluye `optimo`, `camino`, `dp` y `pasos`. Es un modelo multi-etapas que evalúa decisiones de estado y soporta restricciones de valor mínimo para cada etapa.

5.2.6. Método `camino_dag`

La recurrencia para rutas en grafos acíclicos dirigidos es:

$$\text{dist}[v] = \text{mín} / \text{máx} \{ \text{dist}[u] + w(u, v) \}$$

Su salida incluye `optimo`, `camino`, `dp` o `dist`, y `pasos`. El método detecta ciclos, devuelve error si el grafo no es DAG y usa orden topológico para procesar cada vértice una sola vez.

5.2.7. Muestra de prueba del software

Ingreso de datos: EL primer paso es al ingreso de datos, que incluye la selección del método a usar, el objetivo de maximizar y minimizar, el número de etapas, y el número de recursos se prepara una tabla para ingresar los valores por el valor de cada elemento, y dependiendo del algoritmo se puede ver restricciones para valores mínimos que debe tener cada etapa.

ración

Maximizar

Mochila 0/1

Mochila Ilimitada

Tabla Personalizada

Camino en DAG

Etapas:

tal (W):

Generar campos

Objetivo / Datos

Configuración

Tipo:

Maximizar

Algoritmo:

Tabla Personalizada

Nº Ítems / Etapas:

3

Recurso total (W):

10

Generar campos

Función Objetivo / Datos

$r[etapa][x]$ - 3 etapas, $x = 0..10$

Ingrese el retorno de asignar x unidades a cada etap

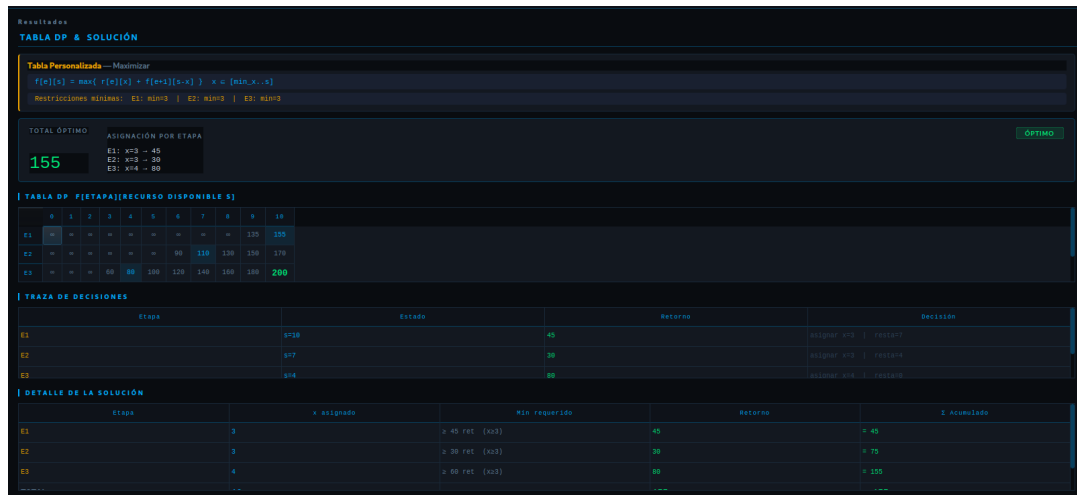
$r[etapa][x]$ - retorno de asignar x unidades a cada e

	x=5	x=6	x=7	x=8	x=9	x=10
E1	70	80	100	120	130	150
E2	50	60	70	80	90	100
E3	100	120	140	160	180	200

Demanda mínima por etapa (en unidades de retorno):

E1	E2	E3
45	30	60

Solución: Para la solución se muestra el proceso etapa por etapa, y las decisiones que se tomaron por etapa. También se muestra el valor según se maximiza o minimiza.



5.3. Complementariedad entre ambos módulos

MetodoDeLaGranM.py resuelve problemas de programación lineal con variables continuas y restricciones lineales. ProgramacionDinamica.py resuelve problemas discretos de optimización combinatoria y de etapas. Ambos buscan optimizar una función objetivo, pero emplean estructuras matemáticas diferentes. Gran M se utiliza cuando el problema puede modelarse como programación lineal con restricciones de igualdad o desigualdad. Programación Dinámica se utiliza cuando el problema tiene estructura discreta, subproblemas superpuestos y decisiones secuenciales.

En el código actual, los dos módulos son independientes: no se invoca MetodoDeLaGranM.solve_gran_m desde ProgramacionDinamica.py, y no hay función compartida ni intercambio directo de datos. En un sistema más amplio, Gran M podría usarse para problemas de mezcla de producción, transporte o formulaciones lineales, mientras que Programación Dinámica serviría para problemas tipo mochila, rutas, planificación por etapas o decisiones discretas.

5.4. Resultados obtenidos por cada algoritmo

5.4.1. Resultados de la Gran M

Cuando la solución es óptima, el módulo devuelve los valores de $x_1, x_2, \dots, x_n$, el valor óptimo Z^* , holguras o excesos y el estado de la solución. Los estados posibles son optimal, infeasible, unbounded y max_iter.

```
{
    'status': 'optimal',
    'sol': {'x1': 12, 'x2': 4},
```

```

    'z_real': 100,
    'slack_sol': {'s1': 0, 'e2': 5},
    'orig_vars': ['x1', 'x2'],
    'maximizar': True,
}

```

5.4.2. Resultados de Programación Dinámica

Para `mochila_01`, la salida típica incluye `optimo`, `seleccionados`, `peso_usado`, `dp` y `factible`.

```

{
    'algoritmo': 'Mochila 0/1',
    'optimo': 25,
    'seleccionados': [0, 2],
    'peso_usado': 7,
    'capacidad': 10,
    'factible': True,
}

```

Para `mochila_ilimitada`, la salida típica contiene el valor óptimo para la capacidad total, la frecuencia de cada ítem y el vector de valores por capacidad.

```

{
    'algoritmo': 'Mochila Ilimitada',
    'optimo': 30,
    'usados': {1: 3, 2: 1},
    'factible': True,
}

```

Para `tabla_personalizada`, se reporta el valor final de la tabla por etapas, el camino de estados y la matriz de resultados parciales.

```

{
    'algoritmo': 'Tabla Personalizada',
    'optimo': 42,
    'camino': [2, 1, 0],
    'factible': True,
}

```

Para `camino_dag`, se reporta el costo o distancia óptima, la ruta desde el origen hasta el destino y el vector de distancias calculadas.

```

{
    'algoritmo': 'Camino en DAG',
    'optimo': 12,
}

```

```

'camino': [0, 3, 5],
'factible': True,
}

```

6. Resultados y Análisis

Los modelos matemáticos presentados en las secciones anteriores fueron ejecutados sobre el problema de distribución de **Logística Distrital S.A.** en las localidades de Usaquén, Chapinero y Kennedy. A continuación se presentan los resultados obtenidos por cada método, junto con su interpretación operativa y una comparación de los enfoques empleados.

6.1. Resultados del Método de la Gran M

Solución óptima obtenida

El algoritmo simplex con la técnica de la Gran M convergió al estado **optimal** luego de las iteraciones presentadas en la Sección 5.1. La solución encontrada asigna los vehículos de la siguiente manera:

Zona	Nombre	Vehículos (x_i^*)	Paquetes entregados	Demanda mínima (d_i)
1	Usaquén	3	$15 \times 3 = 45$	45
2	Chapinero	3	$10 \times 3 = 30$	30
3	Kennedy	3	$20 \times 3 = 60$	60
Total		9	135	135

El valor óptimo de la función objetivo, que representa el tiempo total ponderado de operación de la flota asignada, es:

$$Z^* = 60(3) + 45(3) + 90(3) = 180 + 135 + 270 = \mathbf{585} \text{ minutos}$$

Análisis de la solución

La solución óptima presenta las siguientes características relevantes:

- **Cumplimiento exacto de demandas mínimas:** las tres zonas satisfacen sus compromisos de entrega de forma ajustada, con holgura cero en cada restricción de demanda

($a_1 = a_2 = a_3 = 0$). Esto indica que no existe capacidad ociosa en ninguna zona; cada vehículo asignado contribuye directamente al cumplimiento del contrato.

- **Uso parcial de la flota:** se emplean 9 de los 10 camiones disponibles, lo que deja un vehículo sin asignar. Este resultado es consecuencia directa de minimizar el tiempo total: añadir el décimo camión a cualquier zona incrementaría Z sin reducir el incumplimiento de demanda, dado que ésta ya está cubierta.
- **Distribución uniforme:** el método asigna 3 vehículos a cada zona, lo cual refleja que la combinación de tiempos de viaje y capacidades de entrega por zona produce un equilibrio natural cuando se minimiza el tiempo total bajo las restricciones dadas.
- **Zona Kennedy como cuello de botella:** aunque recibe el mismo número de vehículos que las demás zonas, Kennedy aporta el mayor costo unitario de tiempo (90 min) y a su vez tiene la mayor capacidad de entrega por vehículo (20 paquetes). Esto la convierte en la zona más costosa en tiempo pero también en la más eficiente en volumen, balance que el modelo resuelve asignándole exactamente los vehículos necesarios para cubrir su demanda mínima.

6.2. Resultados del Modelo de Programación Dinámica

Solución óptima por etapas

El modelo de Programación Dinámica resolvió el problema de manera secuencial, tomando decisiones zona por zona con un estado inicial de $s_1 = 10$ vehículos disponibles. Los resultados por etapa fueron:

Etapla (n)	Zona	Vehículos (x_n^*)	Paquetes entregados	Estado (s_n)	$f_n(s_n)$
1	Usaquén	3	$15 \times 3 = 45$	10	45
2	Chapinero	3	$10 \times 3 = 30$	7	75
3	Kennedy	4	$20 \times 4 = 80$	4	155
Total		10	155	—	—

El valor óptimo total de la función de beneficio, medido en paquetes entregados, es:

$$f^* = f_1^* + f_2^* + f_3^* = 45 + 30 + 80 = \mathbf{155} \text{ paquetes}$$

Análisis de la solución

- **Uso total de la flota:** a diferencia del Método de la Gran M, la Programación Dinámica agota los 10 vehículos disponibles, dado que su objetivo es maximizar el volumen de entregas. El recurso adicional (el décimo camión) es redirigido hacia Kennedy, la zona de mayor rendimiento por vehículo.
- **Preferencia por Kennedy en la etapa final:** la asignación de 4 vehículos a la Zona 3 en la tercera etapa se explica por su mayor productividad marginal: cada camión adicional en Kennedy genera 20 paquetes entregados, frente a 15 en Usaquéen y 10 en Chapinero. La ecuación de recurrencia identifica esta zona como la de mayor retorno, y por ello concentra allí el recurso sobrante.
- **Decisiones secuenciales y estado residual:** el estado disponible decrece progresivamente de $s_1 = 10$ a $s_2 = 7$ y $s_3 = 4$, reflejando la asignación acumulada. Este mecanismo garantiza que en ningún momento se supere la disponibilidad de flota, sin necesidad de una restricción global explícita.
- **Cumplimiento de mínimos garantizado:** las demandas mínimas de cada zona (45, 30 y 60 paquetes respectivamente) son satisfechas en todas las etapas. La etapa 3 supera su mínimo, entregando 80 paquetes en lugar de los 60 requeridos, aprovechando el recurso residual disponible.

6.3. Comparación entre modelos

Criterio	Gran M (Minimización)	Prog. Dinámica (Maximización)
Objetivo	Minimizar tiempo total	Maximizar paquetes entregados
Vehículos usados	9 de 10	10 de 10
Asignación Usaquéen	3	3
Asignación Chapinero	3	3
Asignación Kennedy	3	4
Total de paquetes	135	155
Valor objetivo	$Z^* = 585$ min	$f^* = 155$ paquetes
Demandas mínimas	Exactamente cubiertas	Cubiertas y superadas en Kennedy

Ambos modelos coinciden en la asignación de 3 vehículos a Usaquéen y Chapinero, lo que sugiere que esta distribución es robusta independientemente del criterio de optimización. La diferencia principal radica en la Zona Kennedy: cuando el objetivo es minimizar el tiempo, asignar un cuarto camión a Kennedy elevaría innecesariamente Z sin mejorar el cumplimiento

de demanda; cuando el objetivo es maximizar entregas, ese mismo camión produce el mayor beneficio marginal posible, de modo que su asignación a Kennedy es la decisión óptima.

Este contraste ilustra cómo la elección del modelo matemático no es neutral: define qué se considera una solución mejor y, en consecuencia, qué decisión operativa se recomienda. Para **Logística Distrital S.A.**, el Método de la Gran M es adecuado cuando la prioridad es la eficiencia operativa en tiempo y costo de desplazamiento, mientras que la Programación Dinámica resulta más apropiada cuando se busca maximizar el volumen de servicio aprovechando la totalidad de los recursos disponibles.